

Final Report

<https://github.com/leiah8/Capstone2025>

Version 0
Group 22
COMPSCI 4ZP6

Dr. Mehdi Moradi
April 7, 2026

Andre Menezes	meneza3@mcmaster.ca	400401293
Leiah Nay	nayl@mcmaster.ca	400362166
Martin Matlok	matlokm@mcmaster.ca	400369490
Nicholas Poulidis	poulidin@mcmaster.ca	400380146
Tony Lin	lint50@mcmaster.ca	400388309

Complexity

Matching Algorithm

The main complexity of our project comes from our custom matching algorithm. We developed two directions of matching which use the same underlying logic. One ranks candidates for a given project, while the other ranks projects for a given user. Both directions generate the given candidate lists by computing a composite score using three weighted components. These components are semantic similarity, skill match, and interest alignment. There is also an additional additive which Elo-based adjustment applied on top of the weighted sum.

Firstly, semantic similarity between a user's profile, which is populated by the resume parser and the user's inputs, and a project description is computed using a pre-trained sentence transformer model. We evaluated several models and found that the all-MiniLM-L6-v2 model provided the best balance of accuracy and inference speed for our use case. This model generates 384-dimensional¹ vector embeddings of both the given user's bio and a given project description, then computes the cosine similarity between them to produce a score in the range of 0 and 1. One non-trivial edge case that was handled in the development of this algorithm was that if the user has not yet populated their profile, rather than producing a zero score, the engine constructs a proxy bio from the user's listed skills and interests so that even sparse profiles receive meaningful semantic signals. Embeddings are cached using a Redis-backed store such that repeated requests for the same profiles and projects do not re-invoke the model, preventing long wait times for users on loading screens.

Then we introduce a simpler but highly effective component, the skill match and interest alignment. The skill match is the highest weighted component and is calculated through the proportion of a project's required skills that a candidate possesses which produces a ratio in the range of 0 and 1. If a project lists no required skills then we default to a neutral 0.5 rather than a penalising score of 0 for an incomplete project listing. The interest alignment, similarly, compares the user's listed interests against the tags of a given project. The comparison for interests is more complex than a simple string equality check since profile interests and tags are more ambiguous. Instead, through pre-processing of the text, such as applying NLTK's PorterStemmer and normalizing separators, we prevent surface-level formatting differences from artificially lowering match scores. For example, this implementation prevents our matching algorithm penalizing matches in which variations like "Full Stack", "full-stack", and "fullstack" are used. As with the skill matching, the interest alignment score defaults to 0.5 when data is missing and will always fall within the range of 0 and 1.

Finally, we combine these components with the entity's elo. Elo is a dynamic reputation indicator that updates every time someone likes, dislikes, or matches with an entity. It indicates how users view each candidate or project relative to the other candidates and projects in our database. The final Elo-score adjustment is calculated using a sigmoid function centred on the population mean of all entities' Elo ratings in the current request. This implementation ensures that the adjustments are relative to other users or projects rather than a fixed baseline score and it prevents the population Elo scores from being an overall negative addition as the overall positive swipe rate is projected to be low for any given user. Additionally, the adjustments are clamped to be plus or minus 0.5 to ensure Elo adjustments never contribute more to the total weight than our weighted summation that includes skill and semantic similarity. This ensures that a candidate with a strong skill match will still rank above a high-Elo but poorly qualified candidate.

All of these components are combined as a weighted sum to provide a composite score, falling in the range of 0 and 1, for each potential match.

¹ <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>

As the platform had no production swipe data to use for the evaluation of the algorithm as we were creating it, we validated the algorithm by comparing our baseline to results generated using a synthetic LLM-judged dataset² of 900 pairs created with 60 synthetic profiles and 15 synthetic projects which were labelled by Claude Haiku. From this we evaluated four different systems which were a full production model including our Elo-score addition, our model which uses only our weighted summation without Elo, a skills-only comparison algorithm, and a random baseline averaged over 50 random seeds. The full model achieved an AUC of 0.824, the weighted summation only achieved 0.807, skills-only at 0.695, and our random baseline at approximately 0.500. This clearly shows that our full model succeeded at contributing meaningful signals that a pair would be a good match beyond our baselines or basic skill comparison algorithms, with Elo also providing a meaningful boost. Additional evaluation methods such as Precision@K, and NDCG@K supported these results.

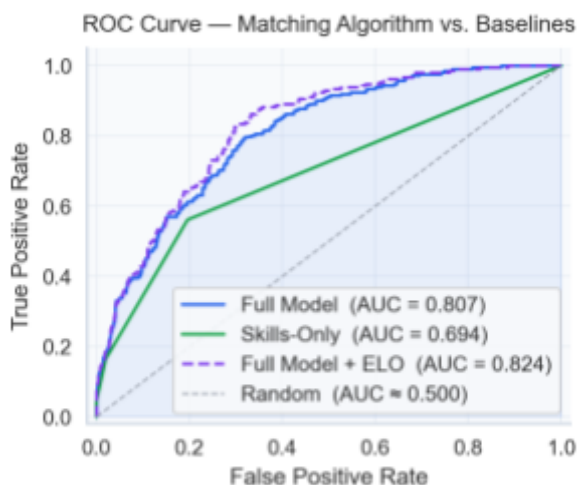


Figure 1. The ROC-AUC for the different algorithms tested.

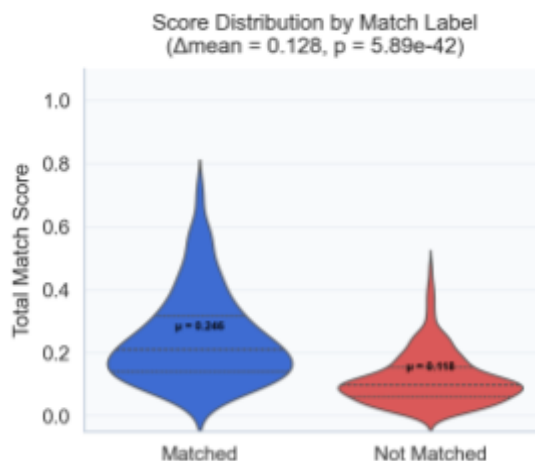


Figure 2. The score distribution for matched vs non-matched pairs by our full algorithm

Resume Parser

The resume parser component introduces a significant layer of technical complexity to the platform. Upon registration and at any time during their experience, users may upload a PDF resume. The system processes this document through a multi-stage pipeline: raw file ingestion, structured information extraction via a large language model, post-processing normalization, and final integration into the matching algorithm's data model.

The complexity of this component comes from several compounding challenges. Resume documents are inherently unstructured and lack a standardized format, where candidates vary widely in how they present education, work experience, skills, personal projects, and biographical information. The parser must reliably identify and extract eight distinct profile attributes (skills, interests, work experience, education, personal projects, a short biography, important links, and location) across this diverse range of document structures. Extracted data is then put through a normalization pipeline that enforces field-level constraints, deduplicates entries across all structured categories, sanitizes OCR artifacts and formatting debris, and validates URL formats, ensuring the output conforms to the strict schema expected by the matching algorithm. A strict JSON schema with enforced output constraints is used to ensure that the model's responses are structurally valid and do not contain hallucinated or fabricated information. Multiple approaches to text extraction and section identification were evaluated during development, including manually implemented parsing algorithms.

² Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, et al. NeurIPS 2023. arXiv: 2306.05685

However, a language model-based extraction approach yielded the most robust and accurate results across the breadth of real-world resume formats encountered.

Achievements and Highlights

We're proud that all P0 and P1 features are implemented with little to no noticeable bugs, with edge case bugs being noted down for future fixing. Our rigorous manual testing proved useful. In terms of subjective highlights, we are quite proud at how cleanly everything came together. Given that our team had limited iOS/app development experience, we are happy with our app as a whole. It is responsive, snappy, and has a clean visual identity.

Additionally, we achieved an accuracy of 80.7% with our custom matching algorithm. With this being the most complex aspect of our project, we are proud to achieve such a high accuracy on such a subjective problem.

Importantly, we believe that our app truly does fit our original vision: an easy-to-use, casual app for finding people and projects. Comparing our app to the "off-the-shelf" solutions mentioned in our SRS reinforces that feeling. Compared to LinkedIn and Indeed, we have successfully differentiated our application by focusing on ease of use and a clean, simple UI. We believe that if launched, people looking for a less intense / more casual experience would reach for our app over the competitors.

Lowlights and Lessons Learned

We had initially planned to use AWS to run our matching algorithm. However, our attempts to use this service were unsuccessful, and our team pivoted to using Railway instead. This goes to show that things will not always go as planned, and we will learn to better prepare for pivots in the future.

One of the more instructive challenges encountered during development was the selection of an appropriate approach for resume parsing. In theory, the problem would be straightforward since resumes, while visually diverse, were expected to follow predictable structural patterns that a conventional OCR and regex-based pipeline could reliably identify. In practice, this assumption proved to be wrong. When using real-world resumes, we understood that they have a far greater degree of structural and formatting variability than anticipated, and the performance of simpler extraction approaches degraded significantly depending on the specific document being processed. The most common issues included headers being inconsistently labelled, failing to capture all significant items, and inconsistent data extraction organized in a row. This experience showed the team the importance of validating assumptions against real input data early in the development cycle, rather than reasoning about feasibility in the abstract. The lesson learned was that unstructured document parsing is deceptively complex, and that investing time upfront to evaluate multiple approaches across a diverse sample of real documents would have avoided a pivot later in the project.

Another lowlight is our enthusiasm for the project. In the beginning, we were quite excited about this idea. Now, while we still strongly believe that it would succeed if launched, none of us have much personal interest in using it. We all already have strong resumes and projects, so we all fall outside of the primary stakeholders. This lack of enthusiasm made working on this project far more difficult than anticipated. In the future, we will aim to find projects where we are fully invested in using the final solution, as this will make it far easier to work on. This is an extremely valuable experience to have now, as during our job search post-graduation, we'll remember the feeling of working on something we aren't super passionate about.

Not all lessons are negative however; We learned a great deal about cooperation when working on software, and how to use each person's strengths to create the best possible outcome. While teaching each other about the skills we are best at, we actively learned from each other as well. All group members are walking out of this project with far more comfort in frontend development, app development, database development, server usage, git workflows, and quality assurance. We learned that a good team is far more important than a good project.

Requirement Completion

Ranking	Requirement	Completion
P0	Front-End Requirements: App Development	
P0	Project browsing feed to look through the various projects available for collaboration	Completed.
P0	- People browsing feed to look through people who want to collaborate - Must have at least one project on your profile to look for people	Completed.
P0	Account sign-up and sign-in pages	Completed.
P0	Project creation page to post a short description of your project idea	Completed.
P0	- Post and profile interactions - Liking and disliking profiles or projects - If a user likes your project, and you like their profile, you will be forwarded their contact information to begin collaborating	Completed. Contact information has been replaced by in-app messaging.
P0	Likes page to see who has liked your profile or specific project	Removed. Instead, you can only see the people you matched with.
P0	Matches page to see the people and projects you have matched with, and their contact information	Completed.
P0	Resume parsing page for users to upload their resume	Completed.
P0	Matching Algorithm:	
P0	- Matching projects/profiles based on the highest relevance score using an NLP-based semantic similarity, with explainable scoring and tunable weights - Parse a resume to analyze job and project descriptions to populate the user profile. - Allow the user to upload any additional projects or skills to their profile - Utilize an NLP algorithm to analyze similarity and compute scores and a ranked list of the most similar projects/profiles.	Completed.

	○ Display the top-matched projects/profiles on respective feeds.	
P0	● Elo matching system for profiles and projects ○ Elo scores will increase or decrease based on interactions with a user's profile or project (i.e. the more likes a profile gets, the higher the elo score) ○ We will be using a project optimal matching system	Completed.
P0	Resume Parsing: ● User can upload their resume which will be parsed to fill in their initial profile ● Preexisting NLP software will be used to identify skills and skill level	Completed.
P0	Database Requirements:	
P0	● Database setup to store project and user profile data ○ Profiles include name, profile pictures, location, mentor/mentee option, education, a brief portfolio with links and descriptions, work experience, interests, skills and skill level (decided by the user on a five-point scale) ○ Projects include name, group size, description, skills needed with minimum skill level, project intention (school project, resume builder, start up), topic(s) of interest, and location	Completed. See Supabase.
P0	● APIs for the client-side application to retrieve and edit their profiles and projects	Completed.
P0	● APIs for the server to access profiles and projects	Completed.
	The client will reach out to the server for profile/project recommendations, the server will compute these recommendations using the matching algorithm and available profiles/projects from the database.	Completed.
P1	Filtering on projects and people ● Capability to filter candidate profiles based on a specific project ● Ability to filter based on skill level, skills, and interests	Mostly Completed. Cannot filter on skill level.
P1	Profile and project customization	Completed.
P1	Messaging system to communicate after matching	Completed.
P1	Allow multiple projects per person	Completed.
P1	Ability to delete or pause (temporarily remove from matching feeds) your project or profile	Completed.

Overall, we completed 100% of our P0 requirements and 97% of our P1 requirements. We also found an alternative solution for our one P2 requirement. Originally, we wanted to tag candidates with which of your projects they would best fit. However, we found it much cleaner and intuitive to force users to browse candidates for a single project at a time. Finally, we completed one of our P3 requirements. We have the resume parser auto-fill the user's profile with skills.